Performance Evaluation of Graphics API Migration from OpenGL to DirectX in Real-Time Volcanic Plume Visualization

A Thesis Proposal

presented to

the Department of Software Technology
College of Computer Studies
De La Salle University

In partial fulfillment
of the requirements for the degree of

Bachelor of Science in Computer Science
Major in Software Technology

by

BERNANDINO, Adrian Rafael
BORROMEO, Anton Miguel
GALAN, Russell Emmanuel
GANAYO, Reinman Geoffrey

Neil DEL GALLEGO
Adviser

July 3, 2026

Abstract

Effective real-time visualization of dynamic natural hazards like volcanic eruptions plays a crucial role in disaster preparedness and public education, yet traditional scientific simulations often compromise real-time interactivity due to high computational overhead. While existing interactive tools like the OpenGL-based AnitoPlume simulator achieve real-time rendering, their reliance on high-level graphics APIs introduces significant CPU driver overhead and limits explicit control over modern GPU resources. This study addresses the challenge of modernizing scientific visualization systems by migrating the rendering subsystem of AnitoPlume to a low-level explicit graphics API, exploring whether the increased implementation complexity yields practical architectural and performance benefits. The primary contribution of this research is a modernized, DirectX-based rendering framework for the AnitoPlume simulator that preserves original physical behaviors and visual output while exposing direct hardware control. To achieve this, the methodology involves systematically porting the rendering pipeline, shader programs, and memory management structures from OpenGL to DirectX, followed by a comparative performance evaluation under identical hardware configurations. This research establishes an empirical baseline for graphics API migration in scientific visualization, providing a highly scalable foundation for integrating modern hardware-accelerated rendering techniques.

Keywords: AnitoPlume, real-time visualization, DirectX, OpenGL, graphics API migration

Contents

Chapter 1

Introduction

1.1 Background of the Study

Volcanic eruptions are among the most destructive natural hazards, producing ash plumes, pyroclastic flows, and volcanic gases that threaten nearby communities and critical infrastructure. Effective visualization of these hazards plays an important role in disaster preparedness, scientific communication, and public education. Traditionally, volcanic simulations have relied on numerical models that prioritize scientific accuracy for hazard prediction and analysis. While these models provide valuable quantitative information, they are often computationally intensive and lack the interactive capabilities necessary for real-time visualization (Rossant & Rougier, 2021).

To overcome these limitations, the sub-field of real-time computer graphics has increasingly been applied to scientific visualization, balancing visual plausibility with high interactive performance. Within this domain, a notable localized development is AnitoPlume, a real-time three-dimensional volcanic plume simulator designed specifically for visualizing the complex eruption dynamics of Taal Volcano (Del Gallego et al., 2026). Developed using C++ and the OpenGL graphics API, this specialized application combines a Lagrangian particle-based plume system with a detailed digital elevation model of the volcano's terrain. By maintaining an interactive rendering performance of approximately 30 frames per second, the system successfully demonstrated that computer graphics could bridge the gap between complex geological events and accessible hazard awareness software for local education and planning (Del Gallego et al., 2026).

Previous efforts in graphics-driven volcanic visualization have heavily relied on

traditional higher-level APIs like OpenGL due to their broad driver portability and structural simplicity (Shiraef, 2016). In these legacy architectures, the graphics API simplifies software development by delegating critical hardware management tasks—such as memory allocation and command scheduling—directly to the vendor's graphics driver. However, as visualization workloads scale, this abstraction inevitably introduces massive CPU driver overhead and restricts developers from optimizing how the hardware executes commands (Bossard, 2019). Consequently, the computer graphics industry has seen a global shift toward modern, low-level explicit graphics APIs like DirectX (Capodieci, Cavicchioli, & Marongiu, 2022), which strip away driver intervention and grant developers granular, direct control over GPU resources, pipeline states, and memory synchronization to maximize hardware efficiency (Ioannidis & Boutsi, 2020).

Despite the theoretical performance advantages offered by modern explicit rendering pipelines, a significant challenge remains in the software modernization of established scientific tools. High-level visualization platforms like the original AnitoPlume simulator are tightly coupled to OpenGL architectures, creating a rigid barrier to optimization and preventing the integration of advanced, contemporary rendering techniques. Transitioning an active simulation system to a low-level framework like DirectX introduces extreme architectural complexity, requiring a complete restructuring of shader programs, command submissions, and memory allocation strategies (Unterguggenberger, Kerbl, & Wimmer, 2022). It remains an open question whether the practical rendering efficiency, resource utilization, and extensibility gains of such a migration justify the steep technical overhead required to port real-time hazard visualization software without altering its underlying scientific behaviors.

1.2 Research Objectives

The primary aim of this study is to modernize the rendering architecture of the AnitoPlume volcanic plume simulator by porting its OpenGL implementation to DirectX and evaluating the impact of the migration on rendering performance, system architecture, and future extensibility.

The specific objectives of this research are as follows:

1. To port the rendering subsystem of the published AnitoPlume simulator from OpenGL to DirectX 12 while strictly preserving its baseline simulation functionality and visual output;
2. To identify and document the technical engineering challenges encountered

during the migration process, focusing on shader translation, rendering pipeline state adaptation, explicit resource management, and command synchronization.

3. To compare the performance of the OpenGL and DirectX 12 implementations under identical hardware environments using empirical metrics, including frame rate (FPS), CPU/GPU resource utilization, and frame rendering times; and

4. To assess the architectural maintainability and extensibility of the new DirectX 12 foundation for integrating future hardware-accelerated rendering techniques into the simulator.

1.3 Scope and Limitations of the Research

This study is bounded by the technical objectives of migrating an existing graphics application to a modern architecture, establishing strict constraints to isolate rendering engine efficiency.

The architectural analysis is limited to comparing the state-machine model of OpenGL with the explicit, low-overhead pipeline model of DirectX 12 within real-time scientific visualization frameworks. This evaluation is strictly constrained to the Windows operating system environment. It excludes alternative modern APIs like Vulkan or Metal because the baseline AnitoPlume application is native to Windows, justifying DirectX 12 as the most appropriate target platform for native hardware acceleration.

The implementation phase is strictly confined to porting the rendering subsystem of the published AnitoPlume simulator from OpenGL to DirectX 12. The scope includes migrating existing graphics resources, translating shader programs into High-Level Shader Language (HLSL), and rewriting command buffers to accurately reproduce the original terrain and particle rendering. This study explicitly excludes any modifications to the core scientific simulation algorithms, Lagrangian physics math, or the terrain mesh generation developed by Del Gallego et al. This boundary ensures that the physical simulation remains a controlled variable, preventing data contamination during the performance comparison.

The documentation of technical challenges is focused entirely on low-level engineering hurdles native to explicit API migration, such as pipeline state object (PSO) compilation and explicit memory synchronization. The study excludes user-facing software usability, user interface design adjustments, or human

participant testing, as application usability was already verified in the baseline study. Instead, data sources are purely technical, derived from automated profiling tools (e.g., PIX or RenderDoc) capturing frame rate (FPS), rendering times, and CPU/GPU utilization during a standardized 180-second simulated eruption sequence. Testing is bounded to standard consumer-grade discrete graphics hardware to reflect the deployment environment of local disaster-response centers.

Finally, the assessment of future extensibility is limited to structural code evaluations and modularity analysis regarding how the new DirectX 12 architecture prepares the simulator to support contemporary rendering pipelines. The actual implementation of advanced features, such as hardware-accelerated ray tracing or compute shaders, is excluded from this study to avoid scope creep while providing a clear architectural roadmap for future researchers.

1.4 Significance of the Research

This study contributes to both computer graphics research and scientific visualization by examining the modernization of an existing real-time simulation system rather than developing an entirely new application. Through the successful migration of AnitoPlume from OpenGL to DirectX, the research provides practical insights into graphics API migration and software modernization for visualization systems.

For researchers in computer graphics, the study offers a detailed case study demonstrating the architectural modifications required when transitioning between two fundamentally different graphics APIs. The comparison between the OpenGL and DirectX implementations contributes empirical evidence regarding rendering performance, implementation complexity, and software maintainability.

For developers of scientific visualization software, the findings may serve as a reference for future projects that require modernization of legacy rendering systems while preserving existing simulation logic. The documented migration process may also assist developers considering similar transitions to lower-level graphics APIs.

For future researchers, this work establishes a modern DirectX-based rendering foundation that may support the integration of advanced graphics techniques, including hardware-accelerated ray tracing, compute shaders, GPU-driven rendering, and other contemporary visualization methods. Consequently, the study extends the long-term applicability of the original AnitoPlume project beyond its initial OpenGL implementation.

Chapter 2

Related Works

2.1 Test

2.2 AnitoPlume

AnitoPlume is the first interactive, real-time 3D graphics-based simulation application explicitly tailored for visualizing the changing landscape of the Taal Volcano island in the Philippines and its corresponding eruption plumes. Developed primarily to address the challenges of volcanic hazard and crisis communication, the simulator serves as an immersive, lightweight alternative to traditional 2D heatmaps and highly complex, non-interactive geoscientific numerical solvers (such as FALL3D and Tephra2). While geoscientific solvers excel at quantitative predictive accuracy, they require heavy high-performance computing resources and significant processing time, making them unviable for applications requiring immediate visual feedback, interactive flexibility, and low-latency user immersion. AnitoPlume fills this gap by merging computer graphics techniques with site-specific real-world terrain details to facilitate risk awareness and "what-if" situational modeling (Del Gallego et al., 2026).

2.2.1 Graphics Architecture and System Performance

To bypass the overhead, heavy abstractions, and unneeded general-purpose features of commercial game engines, AnitoPlume was custom-built using a lean, native C++ open-source API stack consisting of OpenGL for graphics render-

ing, Eigen for numerical linear algebra, and dearIMGUI for the user interface layout. This custom application architecture optimizes real-time performance on consumer-grade hardware. By keeping the system dependencies minimal, the simulator successfully achieves real-time interactivity, running at an average rendering rate of 30 frames per second (FPS) on a system with 6 GB of video RAM (VRAM) capped at 2080 x 2080 resolution(Del Gallego et al., 2026).

2.2.2 Usability Design and Interactive Features

The simulator also introduced several usability improvements over previous volcanic visualization systems, including simplified camera controls, interactive parameter editing, real-time plume direction tracking, and contextual information about nearby communities affected by volcanic activity. These additions improved both accessibility and user experience while preserving interactive performance (Del Gallego et al., 2026).

2.2.3 System Validation and Evaluation Metrics

To evaluate the effectiveness of the simulator, Del Gallego et al. conducted quantitative and qualitative assessments. Visual fidelity was measured using the Structural Similarity Index (SSIM) and Root Mean Squared Error (RMSE) by comparing simulated eruption images with real-world photographs and surveillance footage of Taal Volcano. The study reported high visual similarity, indicating that the synthesized plume closely resembled actual volcanic eruptions. In addition, usability testing using the System Usability Scale (SUS) demonstrated that participants generally found the simulator easy to use and suitable for interactive exploration.

2.2.4 Architectural Limitations and Future Trajectories

Despite these accomplishments, the primary contribution of AnitoPlume lies in the design of its visualization framework rather than the underlying graphics API. The study focused on plume simulation, terrain modeling, visual fidelity, and usability evaluation, while the OpenGL implementation served primarily as the rendering platform supporting these objectives. Consequently, the original work did not investigate how the rendering architecture itself might influence performance, resource utilization, maintainability, or future extensibility.

The authors also acknowledged several opportunities for future development, which include:

1. Photorealistic Rendering
2. Expanded Physics-Based Mechanics
3. Image-Based Parameter Estimation
4. Geoscience Data Integration
5. Automated Terrain Synthesis

These recommendations highlight the critical need for a modern rendering architecture and a low-level API like DirectX 12, which are capable of supporting such intensive graphics and parallel computing workloads while maintaining real-time interactive performance.

2.3 API Migration in Rendering Systems

The migration of rendering software between graphics APIs is a well-known challenge in computer graphics. It often involves more than just replacing function calls. A successful port must account for differences in shader languages, resource-binding models, render-target handling, memory synchronization, and execution semantics. In practice, this means that a program cannot simply be translated line by line; instead, the rendering architecture must be reinterpreted in the context of the target API.

Previous work in graphics engineering has shown that API migration is most successful when the original system is structured in a modular way. Systems that separate rendering logic, scene management, and platform-specific code are often easier to port than tightly coupled implementations. This is particularly relevant for research software, where maintainability and clarity are essential. The process of porting from OpenGL to DirectX, therefore, becomes not only a technical exercise, but also an opportunity to improve the software architecture.

2.3.1 Cross-API Shader Translation

A major issue in porting visual effects from OpenGL to DirectX 12 is shader translation. OpenGL workflows usually rely on the OpenGL Shading Language

(GLSL), while DirectX 12 expects the High-Level Shader Language (HLSL) and a more structured offline compilation pipeline. Existing developer practice tends to fall into two categories: manual shader rewriting for tight control over hardware-specific optimizations, or cross-compilation workflows that preserve shader intent while reducing rewrite cost (Capodieci et al., 2022).

For a visually sensitive effect such as AnitoPlume's volcanic plumes, the second approach is highly attractive. Manual translation risks introducing numerical drift and subtle visual discrepancies in lighting and blending logic. This design philosophy is exemplified by modern research rendering frameworks like NVIDIA's Falcor, which architecturally separates program descriptions from backend compilation. By utilizing intermediate representations like SPIR-V, such engines prioritize shader portability over hard-coded backend assumptions. In a migration study, this supports the argument that accurate visual porting is not just about translating syntax from GLSL to HLSL, but about maintaining shader behavior across compilation targets using tool-assisted pipelines like the DirectX Shader Compiler (DXC) or SPIRV-Cross (Kinkor, 2022).

2.3.2 Pipeline State Management Adaptation

A second major challenge is the shift from OpenGL's dynamic state model to DirectX 12's explicit Pipeline State Objects (PSOs). In architectures, OpenGL allows rendering states to be changed incrementally at runtime, with the graphics driver silently inferring and tracking state combinations (Shiraef, 2016). DirectX 12 completely eliminates this hidden driver-side management. Instead, explicit APIs require developers to bake the blend state, rasterizer state, depth-stencil state, primitive topology, and shader kernels into immutable PSOs ahead of time (Bossard, 2019).

The necessity of this structural rewrite is evident in advanced rendering architectures like the Falcor framework. Rather than relying on hidden driver-side state inference, Falcor explicitly aggregates vertex layouts, shader kernels, rasterizer states, and framebuffer descriptions into a single, pre-compiled state object definition. At draw time, the render context binds this monolithic object directly before issuing the call. For plume rendering, adopting this explicit design is critical; even minor mismatches in depth testing or alpha blending across different PSOs can fundamentally alter how overlapping particle billboards accumulate and fade (Del Gallego et al., 2026). Therefore, PSO migration must be framed as a foundational architectural overhaul rather than a simple API substitution (Asplund, 2020).

2.3.3 Resource Synchronization and Memory Barriers

The most technically demanding aspect of the explicit migration process is resource synchronization. In OpenGL, hardware hazards are concealed by the driver, meaning developers rarely need to explicitly manage the transitions between writing to and reading from the same texture or buffer (Unterguggenberger et al., 2022). DirectX 12, by contrast, transfers the burden of memory management entirely to the developer, requiring explicit resource barriers to ensure that GPU accesses occur in the correct deterministic order (Rossant & Rougier, 2021).

This explicit synchronization burden is concretely demonstrated in the Falcor engine, which utilizes a dedicated barrier system to route resources into their correct states before any updates, copies, or draw calls are executed. By manually handling transitions at both the resource and subresource levels, the architecture reinforces that state transitions are a central engine responsibility rather than an invisible driver service. For the AnitoPlume study, this supports a key argument: accurate migration is not just about producing a similar output image, but about guaranteeing that the GPU views each buffer in the correct state at the exact right microsecond. This is essential when updating thousands of particle billboards, as incorrect barrier placement during parallel draw calls leads to race conditions, stale texture reads, flickering artifacts, or catastrophic device crashes (Cheung, 2024).

References

Asplund, J. (2020). *Directx 12: Performance comparison between single- and multithreaded rendering when culling multiple lights* (Unpublished master's thesis). Jönköping University, School of Engineering.

Bossard, A. (2019). High-performance graphics with directx in racket. *Information Engineering Express*, *5*(2), 83–98. doi: 10.52731/iee.v5.i2.368

Capodieci, N., Cavicchioli, R., & Marongiu, A. (2022). A taxonomy of modern gpgpu programming methods: On the benefits of a unified specification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *41*, 1649–1662. doi: 10.1109/tcad.2021.3082863

Cheung, D. (2024). *Applications and limitations of vulkan-layer-based instrumentation* (Unpublished master's thesis). University of Alberta (Department of Computing Science).

Del Gallego, N. P., Torre, O. D., Arquillo, C. M., Cordero, V. V., Sales, J. L., & Yarte, S. L. (2026). Interactive 3d simulation of taal volcano eruption plumes. *Applied Computing and Geosciences*, 100331.

Ioannidis, C., & Boutsi, A.-M. (2020). MULTITHREADED RENDERING FOR CROSS-PLATFORM 3D VISUALIZATION BASED ON VULKAN API. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *XLIV-4/W1-2020*, 57–62. doi: 10.5194/isprs-archives-xliv-4-w1-2020-57-2020

Kinkor, J. (2022). Easy vulkan. In *Proceedings of the 27th student eeict 2022.* Brno University of Technology, Faculty of Information Technology.

Rossant, C., & Rougier, N. P. (2021). High-performance interactive scientific visualization with datoviz via the vulkan low-level gpu api. *Authorea Preprints*. doi: 10.22541/au.162129047.76547134/v1

Shiraef, J. A. (2016). *An exploratory study of high performance graphics application programming interfaces* (Master's thesis, University of Tennessee at Chattanooga). Retrieved from `https://scholar.utc.edu/theses/446/`

Unterguggenberger, J., Kerbl, B., & Wimmer, M. (2022). The road to vulkan: Teaching modern low-level apis in introductory graphics courses. In *Eurographics 2022 - education papers.* The Eurographics Association. doi:

10.2312/eged.20221043